ChatGPT

Executive Summary

- **Strict train–val–test split:** Fix any leakage first. Compute all metrics' normalization, layer weights, thresholds or calibrators **only on TRAIN/VAL** and freeze them for TEST. Never tune on the TEST split [1].
- **Trainable combiner:** Train a simple logistic regression on token-level metric features (cosine, Mahalanobis, logit-lens, PCA, CIE) using TRAIN data. This will learn to upweight strong metrics and downweight noise [2] [1]. Use its probability as the "trained" composite score (with optional Platt calibration on VAL for ECE).
- **Per-layer weighting:** Compute each metric per layer (already done in the code) and learn nonnegative layer weights from VAL: e.g. set each layer's weight B its AUROC on VAL. Use these weights to aggregate per-layer metric scores [1]. This yields a weighted-layer mean instead of a uniform mean.
- **Mahalanobis shrinkage:** Improve `fit_mahalanobis()` by adding covariance shrinkage (e.g. Ledoit–Wolf or simple λI) to ensure invertibility in high dimensions [2] [3]. This can significantly stabilize the distance and raise AUROC, as known from OOD work [2].
- **Robust normalization:** Instead of only z-scores, fit a robust scaler (median/IQR or median/MAD) on TRAIN features for each metric to use in the combiner and composite. This reduces the impact of outliers.
- **Optional smoothing/aggregation tweaks (medium priority):** Consider token-level smoothing for cosine drift (e.g. averaging with neighbors) or alternative aggregations (max vs mean) if needed. Also vary PCA components (e.g. 16 32). These are lower priority optimizations.

Expected impact: The biggest gains come from removing leakage and replacing the naive unweighted composite with a learned combiner plus layer weighting. We expect MAJOR improvements (_ +0.10 AUROC) from the learned combiner and shrinkage, and moderate gains from robust scaling [2] [1]. Other tweaks (smoothing, PCA tuning) have smaller impact.

Architecture: Save and reuse artifacts in a consistent way. E.g., `stats.pt` should contain the mean/cov/PCA from TRAIN, a robust normalizer, a logistic combiner model, learned layer weights, and (optionally) a Platt calibrator. Use command-line flags to select composite mode (`equal|weighted|trained`).

Validation: Use TRAIN to fit all internal parameters and VAL to tune hyperparameters (e.g. layer weights, combiner C). Compute final AUROC (with 95% bootstrap CI) on the TEST split only [1]. Aim for test composite AUROC _0.70 with both baseline metrics beaten for full marks.

Key references: Use established methods for each fix. For Mahalanobis shrinkage see Lee et al. (2018) and follow-ups [2] [4]. For logit-lens see the alignment forums (Nostalgebraist, 2020) and recent analyses [5]. For representation-based OOD/hallucination, see ReDeEP (Sun et al. 2025), probabilistic distance methods [6], and latent-state fidelity approaches [1].

---

1. Metric Tuning Dimensions

We focus on the five non-baseline **representation metrics** from *metrics.py*: **cosine_drift**, **mahalanobis**, **logit_lens**, **pca_deviation**, and **cie_top3**. For each, we enumerate tunable dimensions and how to implement them:

- **Cosine Drift (intra-answer cos distance):** The function `cosine_drift` computes per-layer cosine similarity between each token and its predecessor, then returns the mean over layers.
- **Layer selection/weighting:** The code currently averages all layers. Instead, learn per-layer weights (from VAL AUROC) and compute a weighted mean of `drift[l,t]`. This can highlight the informative layer range (e.g. mid layers). Implementation: apply a custom weighted sum after computing the `(n_layers, n_tokens)` drift matrix (see *weighted_layer_mean* below). Expected impact **medium-high** (it can amplify signal from key layers) [1].
- **Aggregation (max vs mean):** The code averages the 1 ?cos across layers. An alternative is to take the maximum drift across layers (meaning "somewhere I drifted a lot"). Test both. Use VAL to pick: likely *mean* is baseline, *max* could catch any strong signal. Impact likely **low-medium**.
- **Temporal smoothing:** Instead of just adjacent-token drift, one could incorporate a window (e.g. average drift over last 2-3 steps) or even the norm difference `||h_t − h_{t-1}||`. Implement by convolving the time series of drift values (or hidden states). Expected impact **low** (narrow benefit). If used, compute in `cosine_drift` or post-process the returned drift.

- **Normalization:** Cosine drift values 9 [0,2] and have skew. After computing per-token drift, apply a robust scaler (median/IQR) fitted on TRAIN for this metric. This helps the composite combiner use it. Implementation: add to `fit_robust_normalizer` usage. Impact **low-medium** (more stable blending) [2].

- **Mahalanobis Distance:** The code's `fit_mahalanobis` and `mahalanobis_per_layer` measure each layer's distance from the "faithful" (token_label=0) state distribution learned on TRAIN. Then `mahalanobis` averages over layers.

- **Covariance shrinkage / regularization:** The current code adds `λI` with λ=1e-4. Raise this or use Ledoit–Wolf to improve invertibility. For example, use `sklearn.covariance.LedoitWolf` to compute the shrinkage version of Σ for each layer [2]. In code: modify `fit_mahalanobis` to estimate Σ via Ledoit-Wolf (or at least a larger ridge λ). Expected impact **high** (addresses poor-conditioning, a known issue in high-dim Mahalanobis) [2].
- **Dimensionality reduction:** If the hidden dimensionality is very large relative to sample count, perform PCA on each layer first (keeping, say, 32 components) and compute Mahalanobis in that subspace. This is like applying `fit_pca` to shrink dimension. Impact **low** (complicated and overlapping with existing PCA metric).
- **Per-layer weighting:** Instead of simple mean across layers, weight by VAL AUROC. (Recompute AUROC(mahalanobis) per layer on VAL and set layer weights B that.) Implementation: same *weighted_layer_mean* approach. Impact **medium** (mirrors the idea of CIE: focus on top layers) [1].

- **Score transformation:** The raw Mahalanobis distances grow with magnitude. Optionally transform per-layer distances by e.g. z-scoring (subtract train mean, divide by std) or use robust scale. The composite normally does z-score per metric, but per-layer standardization ensures no layer

dominates purely by scale. Implementation: apply a per-layer normalizer inside `mahalanobis_per_layer` or in combiner features. Impact **low** but safe.

- **Logit-lens KL:** `logit_lens(kl_per_layer)` takes precomputed KL divergences from projecting each layer's hidden state to the final logit distribution, then averages.

- **Layer weighting:** The same reasoning applies—compute VAL-layer AUROCs for logit-lens and weight accordingly. This can be done via *weighted_layer_mean*. (We do this anyway for all multi-layer metrics.) Impact **medium** (deep layers often matter more for KL shifts).
- **Aggregation variant:** We may try max-over-layers or picking top-K layers (like "CIE for logit-lens"), but simpler is usually fine.
- **Normalization:** KL values can vary; ensure robust or z-scaling across tokens (done by composite normalizer). Consider sign: since higher KL means more anomaly, likely keep sign=+1.

- **Alternative divergence:** Instead of KL(final||layer), one could use symmetric KL or JS-divergence, but this complicates. Probably low impact since KL already used by prior work (Du et al. 2024).

- **PCA Deviation:** `pca_deviation` projects hidden states onto a PCA subspace (fitted on TRAIN) and measures the residual norm. Tunable aspects:

- **Number of components (k):** Default 16 per layer. Try increasing to 32 or 64—more components make the subspace richer and residuals smaller (possibly lowering signal). Conversely, fewer components might exaggerate anomalies. Use VAL to pick k (between 8–64). Impact **low-medium** (likely minor gains).
- **Weighted mean:** Weight layers by VAL AUROC as above.
- **Normalization:** PCA norms may scale with hidden dimension; apply robust scaler on the output scores.

- **Alternative measure:** Possibly use *relative* deviation (dividing by original hidden norm) so each token is fraction outside subspace. Implementation is straightforward. Impact **low**.

- **CIE Top-3:** `cie_top3` averages Mahalanobis on the 3 best layers (by VAL Mahalanobis AUROC). It's already a tuned version of Mahalanobis. Variables:

- **Number of layers (N):** Try N=2 or 4 instead of 3. Implementation: pick the top N layers on VAL.
- **Weighting vs mean:** Instead of unweighted mean of those layers, weight them by their VAL AUROC (or distance).
- **Different metric for ranking:** Could pick top layers based on logit-lens or PCA AUROC instead of Mahalanobis; but Mahalanobis is most natural here.
- **Normalization:** Same as Mahalanobis.
- **Transform:** Same as Mahalanobis (distance or something).

<figure>

</figure> *Figure: Example layer-profile (attention-entropy baseline) showing per-layer scores for factual (blue) vs hallucinated (red) generations [49]. A similar plot for each metric helps identify which layers carry signal (the separation) and guides weighted aggregation.*

2. Implementation Plan

We make minimal code changes to introduce the above improvements, focusing on high-impact ones.

- `src/metrics.py` **additions:**
- **Robust normalizer:** Add functions

```
def fit_robust_normalizer(X: Tensor) -> dict:
    # Compute per-feature medians and IQR (or MAD)
def apply_robust_normalizer(X: Tensor, norm: dict) -> Tensor:
    # Center by median, scale by IQR (avoid division by zero).
```

Use these when fitting the combiner and for composite if desired. Impact: moderate stability [2].

- **Metric combiner:** Add

```
def fit_metric_combiner(X_train, y_train, C=1.0): ...
def apply_metric_combiner(X, combiner): ...
```

This wraps a sklearn `LogisticRegression(C=C)` (or PyTorch linear) trained on token features (7-dim: one per metric) to predict token label. Return the trained model (serialize it). High impact: learns optimal weights.

- **Weighted layer mean:**

```
def weighted_layer_mean(scores: Tensor, weights: Optional[Tensor]) ->
Tensor:
    # scores: (n_layers, n_tokens). If weights is 1D (n_layers,), return
(weights*scores).sum(0).
    # If weights is None or all ones, fallback to uniform mean.
```

Use this in place of `mean(dim=0)` for metrics. For example, replace `drift.mean(dim=0)` with `weighted_layer_mean(drift, w_cosine)`.

- **Improve** `fit_mahalanobis`**:** Expand to accept a `shrinkage` parameter. For simplicity, add a tunable `λ` default 1e-4; allow passing `shrinkage='LedoitWolf'` to use sklearn if available. Example:

```
cov = cov + (regularization * torch.eye(d)).unsqueeze(0)
inv_cov = torch.linalg.pinv(_finite(cov))
```

could be replaced by Ledoit–Wolf (via `sklearn.covariance.LedoitWolf().fit` per-layer) if implemented. This will be serializable via the packed matrix functions already in code.

- **Combine vs expand normalizer:** The existing `fit_normalizers` (mean/std) remains for composite. Do NOT modify it to use VAL; it must be train-only. We will add robust normalizers separately for combiner inputs.

- `pipeline/2-fit.py` **changes:**

- **Fit on TRAIN only:** Load train artifacts, compute all training-fit stats here (mean/Σ for Mahalanobis, PCA comps, etc). Then load VAL artifacts **only to select hyperparameters** (layer weights, logistic C, number of PCA comps, calibrator). Do not use TEST. Save everything to `stats.pt`. Include:
  Mahalanobis stats: `stats['mahalobis_mean']`, `stats['mahalobis_cov']` or packed inv_cov.
  PCA stats: as before, `stats['pca_mean']`, `stats['pca_components']`.
  Robust normalizers: `stats['robust']={metric_name: {med,iqr}}` (for each metric).
  Layer weights: e.g. `stats['layer_weights'] = {'cosine': [...], 'mahalanobis': [...], ...}`.
  Combiner: `stats['combiner']` = serialized sklearn LR (coef & intercept) or state_dict if PyTorch. Also record which metrics/features used and their robust norm parameters.
  Optional calibrator: If using Platt scaling (Logistic on combiner outputs), save `stats['calibrator']` (sklearn `CalibratedClassifierCV` or parameters).
  Meta: `stats['trained_on'] = 'train'`, `stats['val_data_used'] = 'val'` for clarity.
- **Compute layer weights:** For each metric, iterate layers, compute AUROC on VAL (without using TEST). Set any negative weights to zero, normalize to sum=1. Example: in code, after scoring VAL with each metric, compute `auroc_layer[l]` and do `w = np.clip(auroc_layer, 0, None); w /= w.sum()`.
- **Fit logistic combiner:** Build token-level matrix from training data: columns are the **five** representation metrics (after layer-aggregation via learned weights). Optionally include the two baselines as separate, but for composite we exclude them. Include only *cosine_drift, mahalanobis, logit_lens, pca_deviation, cie_top3*. Fit LR to predict token_label=hallucinated (0/1). Use VAL to pick regularization C (grid search 0.01–10). Expected outcome: strong weight for logit_lens, next for CIE/Mahalanobis, near-zero for weak metrics.
- **Fit calibrator:** If ECE is poor, fit a Platt scaling on VAL (logistic on the LR scores) and store it. This is optional and low priority; ECE is not the mark criterion.

- **Save stats:** Use existing save mechanism to extend `stats.pt` (likely a Torch file). Ensure all new objects (combiner coefficients, normalizer params, weights) are JSON-serializable or torchable.

- `pipeline/3-score.py` **changes:**

- **Load** `stats.pt`**:** At start, load the new keys.
- **No fitting:** Use loaded artifacts only.
- **Layer aggregation:** When computing each metric's per-layer matrix, replace uniform `.mean(dim=0)` with `weighted_layer_mean(..., stats['layer_weights'][metric])` for modes `'weighted'` or `'trained'`. Keep original for `'equal'`.

- **Composite modes:** Add CLI option `--composite-method {equal,weighted,trained}`.
  equal: Original z-scoring + mean of metrics (baseline behavior).
  weighted: Use weighted layer means + equal metric weights (i.e. metrics still averaged equally). This is intermediate.
  trained: After computing each token's scalar metric values (using weighted layers), apply the robust normalizer (fit on TRAIN) then the trained logistic combiner (`apply_metric_combiner`) to get a probability. Optionally apply calibrator. This probability is the composite score.

- **Output:** No change to log format except ensure the composite column reflects the chosen method. Also output individual metric scores (for E1) as before.

- `pipeline/4-eval.py` **changes:**

- **Evaluation-only:** Remove any code that fits thresholds or calibrators. Instead, if threshold-based metrics are needed (e.g. F1), either load thresholds from `stats.pt` (if precomputed on VAL) or report threshold-independent metrics (AUROC). For confidence calibration (ECE), use frozen composite probabilities (no refitting).
- **Split labels:** Ensure it reports **TEST** metrics. The rubric forbids reporting VAL results. Print a note summarizing which split each stat came from (TRAIN/VAL).
- **CI:** Use bootstrap on TEST predictions to compute 95% confidence intervals for AUROC (as recommended by Hallucination Basins [1]).

```
timeline
    title Implementation Roadmap
    section Fix Leakage (Phase 1)
    Enforce TRAIN/VAL/Test splits    :done, 2026-04-01, 3d
    Refactor 4-eval to no fitting   :done, 2026-04-04, 2d
    section Enhancements (Phase 2)
    Add robust normalizer functions :active, 2026-04-06, 3d
    Implement layer-weighted mean   :active, 2026-04-09, 2d
    Add Ledoit-Wolf shrinkage to fit_mahalanobis :planned, 2026-04-11, 4d
    section Combiner (Phase 3)
    Fit logistic combiner on TRAIN   :planned, 2026-04-15, 3d
    Add composite modes (--equal/--trained) :planned, 2026-04-18, 2d
    Optionally: fit calibrator on VAL:planned, 2026-04-20, 1d
    Final test & ablations           :planned, 2026-04-21, 3d
```

3. Hyperparameters & Implementation Details

- **Combiner LR (C):** Try `C` in [0.01,0.1,1,10]. Smaller C = more regularization (could be needed if token labels are noisy).
- **Layer weight normalization:** Use **nonnegative** weights and divide by sum (with epsilon=1e-6).
- **Shrinkage λ:** Try λ 9 {1e-4, 1e-3, 1e-2}. Alternatively use sklearn's Ledoit-Wolf (no hyperparam). Expect larger λ to modestly degrade performance if too high, so tune on VAL if possible. Impact **high** if improving invertibility [2].

- **Robust scaler:** Use median and IQR (or MAD). No hyperparam beyond robust choice (IQR vs MAD).
- **Temporal smoothing (if any):** Weight recent cosines, e.g.

```
drift[:,1:] = 1 - cos(h[:,1:], h[:,:-1])
drift = drift.mean(dim=0)
# as a post-hoc smooth: final_score[t] = (drift[t] + drift[t-1]) / 2
```

Or convolve with [0.5,0.5] kernel. Expect **low** gain; skip if time is short.

4. Evaluation Protocol

- **No leakage:** All fitting on TRAIN, tuning on VAL, final metrics on TEST [1]. In particular, compute `μ_l, Σ_l` on TRAIN only, then use them to score TEST.
- **Bootstrap CI:** For each reported AUROC on TEST, compute 95% CI via bootstrap resampling of the test tokens (as in prior work [1]). Report mean±CI.
- **Metrics:** Report Test AUROC (with CI), F1 at an appropriate threshold (either fixed or threshold-free F1-span), Spearman ρ (ranking correlation), and ECE. Baselines and individual metrics must also be reported (but do not include them in the composite). The rubric's full composite AUROC on TEST (_0.70 for 4 marks) is the primary goal.
- **Thresholding:** If using any threshold-based metric, fix thresholds on VAL only. E.g., choose the DET or ROC point at 95% TPR on VAL to compute F1 on TEST. Do not tune on TEST.
- **Acceptance:** Composite AUROC _0.70 on TEST, with both baselines beaten (i.e. composite > attention-ent, logit-conf) [1]. Also include a layer-profile analysis plot.

5. Ablation Matrix

Intervention	Cosine Drift	Mahalanobis	Logit-Lens	PCA Deviation	CIE Top3
Baseline (equal mean across layers, no combiner)	0.43	0.47	0.65	0.38	0.45
Layer weights (VAL-AUROC)	(med)	(med)	(med)	(low)	(med)
Shrinkage in Mahalanobis	–	(high)	–	–	(high) (via Mahalanobis)
Robust scaling (combiner input)	(low)	(low)	(low)	(low)	(low)
Logistic combiner (trained)	(high)	(high)	(high)	(med)	(high)
Temporal smoothing	(low)	–	–	–	–

Intervention	Cosine Drift	Mahalanobis	Logit-Lens	PCA Deviation	CIE Top3
Increase PCA k (16 32)	–	–	–	(if any)	–
Platt calibration (ECE)	–	–	–	–	–
Max-aggregation (vs mean)	Experimental	/–	–	–	–

(Cells show expected effect on that metric's AUROC when applied. " " indicates likely improvement, " " drop, "–" no direct effect. The combiner affects all metrics indirectly.)

We will first test **Layer weights** and **Mahalanobis shrinkage** individually to see gains (especially for Mahalanobis itself). Then the big leap is **Logistic combiner** (trained), which jointly reweights all features. Platt calibration is only for ECE.

6. Artifacts to Save in `stats.pt`

Ensure reproducibility by saving all fitted parameters from TRAIN/VAL:

- `'mahalobis_mean'`: (n_layers, hidden_dim) tensor of μ for each layer (TRAIN-only).
- `'mahalobis_inv_cov'`: packed upper-triangular of each layer's inverse covariance (after shrinkage).
- `'pca_mean'` and `'pca_components'`: same as before.
- `'normalizers'`: original composite (mean,std) from TRAIN for each metric.
- `'robust'`: a dict of `{metric: {'median':..., 'iqr':...}}` fitted on TRAIN token values for robust scaling before combining.
- `'layer_weights'`: a dict `{metric: weight_vector}` giving per-layer weights (summing to 1) derived from VAL AUROC.
- `'combiner'`: serialized logistic regression (weights + intercept), e.g. a dict with keys `coef`, `intercept`, and maybe `features_order`.
- `'calibrator'`: if used, `{'a':..., 'b':...}` Platt parameters or full sklearn object.
- `'top3_layers'`: the list of 3 layers indices used by CIE (optional if we recompute that logic).
- `'split_info'`: metadata like which split was train/val/test.

Each key should document in comments what it contains.

7. Key References

- **Mahalanobis + shrinkage:** Ledoit & Wolf (2004) show optimal covariance shrinkage in high-dim scenarios [2]. In practice, OOD detectors using Mahalanobis with shrinkage achieve strong AUROC [2] [4].
- **Logit Lens:** Introduced by Nostalgia2020 (interpreting LLM hidden states) and used for probing fact vs hallucination (Du et al. 2024) [5]. The logit-lens KL divergence is a known internal "anomaly score".

- **Representation-based Hallucination Detection:** ReDeEP (Sun et al., ICLR'25) decouples context vs parametric signals, focusing on internal features [7] [8]. The **ReDeEP** framework validates using internal metrics but we depart by learning a combiner. Hallucination Basins (ICLR'26) emphasizes strict train/test splits and bootstrap CI [1].
- **Probabilistic Distances:** Kuleshova et al. (2024) propose MMD-based distances between prompt vs response hidden states [6]. This underscores using hidden-state distributions to detect hallucination.
- **OOD Detection:** Lee et al. (NIPS'18) use Mahalanobis in feature space as a confidence score [4]. This justifies Mahalanobis distance in our metrics.
- **Calibration/ECE:** Common ML practice. (For reference, *Hallucination Basins* used bootstrap CIs and froze stats as above [1].)

8. Actionable Checklist & Commands

- [ ] **Fix leakage:** Ensure `2-fit.py` only fits on TRAIN, and `4-eval.py` does no fitting.
- [ ] **Implement robust normalizer:** Add `fit_robust_normalizer` / `apply_robust_normalizer` in `src/metrics.py`.
- [ ] **Implement weighted layer mean:** Add `weighted_layer_mean` and use it in metrics.
- [ ] **Modify `fit_mahalanobis`:** Add shrinkage (e.g. set larger λ or Ledoit–Wolf).
- [ ] **Modify `2-fit.py`:** After loading TRAIN data, compute layerwise AUROCs on VAL for each metric and save as `stats['layer_weights']`. Build token-level feature matrix, apply robust scaler (TRAIN), fit logistic combiner, save it. Optional: fit Platt on VAL.
- [ ] **Modify `3-score.py`:** Load `stats.pt`, apply layer weights and combiner depending on `--composite-method`. Add CLI flag for method.
- [ ] **Modify `4-eval.py`:** Remove all fitting; load thresholds/calibrators from `stats.pt` if needed. Report TEST metrics with bootstrap CI.
- [ ] **Test & Iterate:** Run on train/val/test splits. Ablate one change at a time to confirm improvements.

Commands: (example sequence)

```
python pipeline/1-infer.py --split train,val,test
python pipeline/2-fit.py --train-split train --val-split val
python pipeline/3-score.py --split train,val,test --composite-method trained
python pipeline/4-eval.py --split test
```

Ensure each step succeeds and that TEST AUROC (trained composite) is _0.70 (with CI) for full credit [1].

---

[1] Hallucination Basins: A Dynamic Framework for Understanding and Controlling LLM Hallucinations
https://arxiv.org/html/2604.04743v1

[2] [3] assets.amazon.science
https://assets.amazon.science/64/48/4d1e599c4472859e27a9f7647786/few-shot-out-of-domain-intent-detection-with-covariance-corrected-mahalanobis-distance.pdf

4 [1807.03888] A Simple Unified Framework for Detecting Out-of-Distribution Samples and Adversarial Attacks
https://arxiv.org/abs/1807.03888

5 arxiv.org
https://arxiv.org/pdf/2410.11414

6 Probabilistic distances-based hallucination detection in LLMs with RAG
https://arxiv.org/html/2506.09886v2

7 8 ReDeEP: Detecting Hallucination in Retrieval-Augmented Generation via Mechanistic Interpretability | OpenReview
https://openreview.net/forum?id=ztzZDzgfrh